- Multilayer perceptron architecture
-

We've seen how a neural network can be designed to have more than one neuron.

The main components of the neural network architecture are as follows:

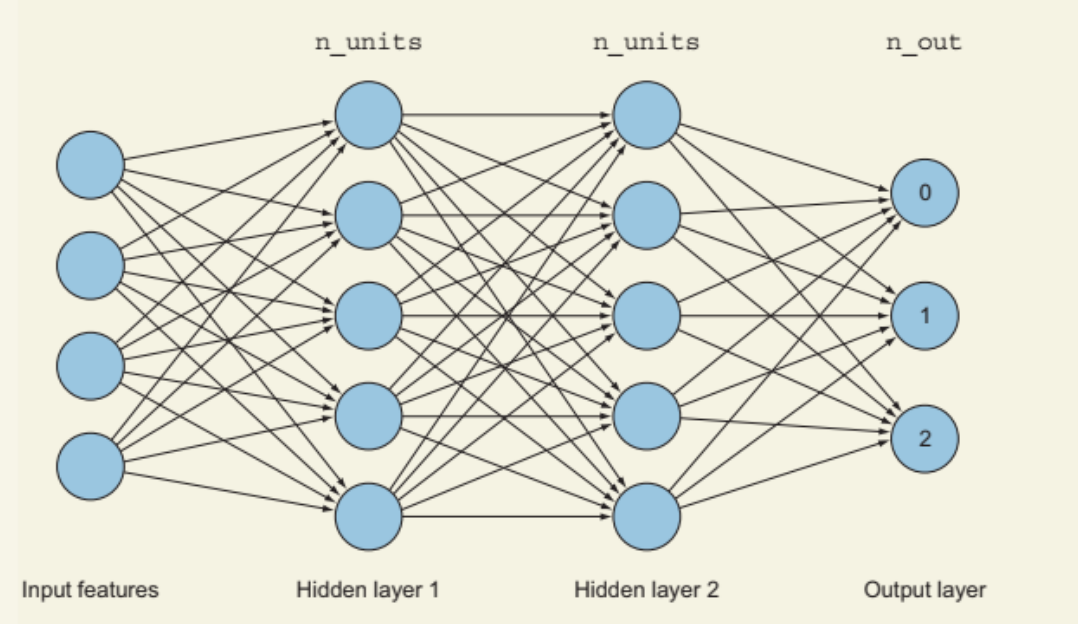
- **Input layer** —Contains the feature vector.

- **Hidden layers** —The neurons are stacked on top of each other in hidden layers. They are called "hidden" layers because we don't see or control the input going into these layers or the output. All we do is feed the feature vector to the input layer and see the output coming out of the output layer.

- **Weight connections (edges)** —Weights are assigned to each connection between the nodes to reflect the importance of their influence on the final output prediction. In graph network terms, these are called edges connecting the nodes.

- **Output layer**—We get the answer or prediction from our model from the output layer. Depending on the setup of the neural network, the final output may be a real-valued output (regression problem) or a set of probabilities (classification problem). This is determined by the type of activation function we use in the neurons in the output layer.

Fully connected layers




```
model.summary()
```

The output will be as follows:

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 5)                 25
_________________________________________________________________
dense_1 (Dense)              (None, 5)                 30
_________________________________________________________________
dense_2 (Dense)              (None, 3)                 18
=================================================================
Total params: 73
Trainable params: 73
Non-trainable params: 0
```

Why hidden layers?

- In neural networks, we stack hidden layers to learn complex features from each other until we fit our data.
- So when you are designing your neural network, if your network is not fitting the data, the solution could be adding more hidden layers.
- **How many layers, and how many nodes in each layer?**
- A network can have one or more hidden layers (technically, as many as you want). Each layer has one or more neurons (again, as many as you want).
- Your main job, as a machine learning engineer, is to design these layers. Usually, when we have two or more hidden layers, we call this a deep neural network.
- The general rule is this:
- The deeper your network is, the more it will fit the training data. But too much depth is not a good thing, because the network can fit the training data so much that it fails to generalize when you show it new data (overfitting); also, it becomes more computationally expensive. So your job is to build a network that is not too simple (one neuron) and not too complex for your data.

Activation functions

- also referred to as transfer functions or nonlinearities because they transform the linear combination of a weighted sum into a nonlinear model. An activation function is placed at the end of each perceptron to decide whether to activate this neuron.

- Why use activation functions at all? Why not just calculate the weighted sum of our network and propagate that through the hidden layers to produce an output?

- The purpose of the activation function is to introduce nonlinearity into the network. Without it, a multilayer perceptron will perform similarly to a single perceptron no matter how many layers we add. Activation functions are needed to restrict the output value to a certain finite value.
- So without the activation function, we just have a linear function that produces a number, but no decision is made in this perceptron. The activation function is what decides whether to fire this perceptron.
- $z = height \cdot w1 + age \cdot w2 + b$; >0 accepted <0 rejected

neural network hyperparameters:

- Number of hidden layers?
- What Activation function I have to use?
- Error (Loss) Function
- Optimizer

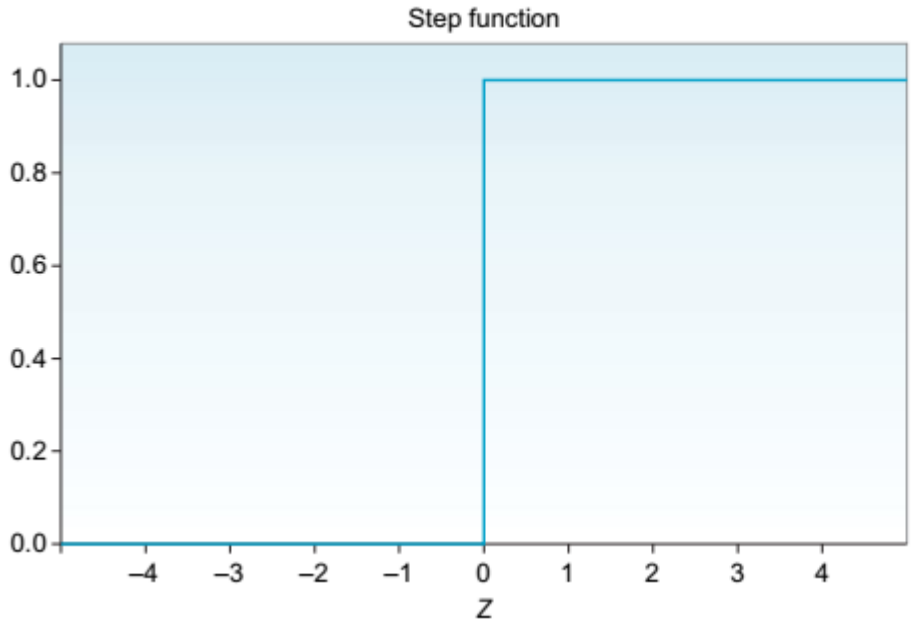
Some of the most common types of activation functions.

- **Linear transfer function**
- also called an identity function, indicates that the function passes a signal through unchanged. In practical terms, the output will be equal to the input, which means we don’t actually have an activation function. So no matter how many layers our neural network has, all it is doing is computing a linear activation function or, at most, scaling the weighted average coming in. But it doesn’t transform input into a nonlinear function.
    - activation(z) = z = wx + b

Some of the most common types of activation functions.

- **step function**

- The step function produces a binary output. It basically says that if the input x > 0, it fires (output y = 1); else (input < 0), it doesn't fire (output y = 0). It is mainly used in binary classification problems like true or false, spam or not spam, and pass or fail.
-



$$\text{Output} = \begin{cases} 0 \text{ If } w \cdot x + b \leq 0 \\ 1 \text{ If } w \cdot x + b > 0 \end{cases}$$

Some of the most common types of activation functions.

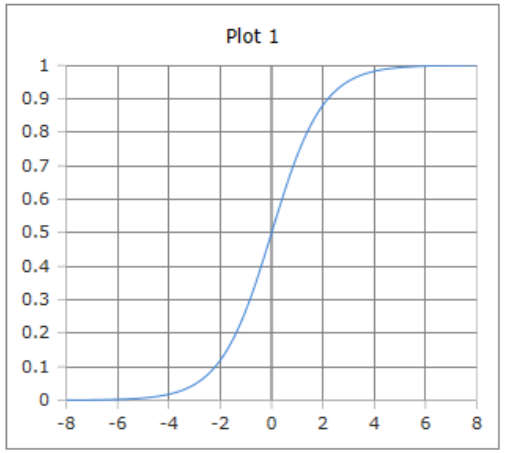
- **Sigmoid/logistic function**

This is one of the most common activation functions. It is often used in **binary classifiers** to predict the probability of a class when you have **two classes**. The sigmoid squishes all the values to a probability between 0 and 1, which reduces extreme values or outliers in the data without removing them.

Sigmoid or logistic functions convert infinite continuous variables (range between $-\infty$ to $+\infty$) into simple probabilities between 0 and 1.

While the step function is used to produce a discrete answer (pass or fail), sigmoid is used to produce the probability of passing and probability of failing.

$$z = wx + b$$




```
model.add(layers.Dense(64, activation=sigmoid'))
```

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

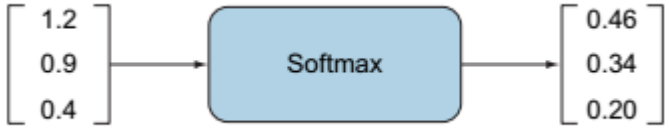
Some of the most common types of activation functions.

- The **softmax** function is a generalization of the sigmoid function.
- It is used to obtain classification probabilities when we have more than two classes. It forces the outputs of a neural network to sum to 1 (for example, 0 < output < 1).
- A very common use case in deep learning problems is to predict a single class out of many options (more than two).

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$

- Softmax is the go-to function that you will often use at the output layer of a classifier when you are working on a problem where you need to predict a class between more than two classes. Softmax works fine if you are classifying two classes, as well.

```
layer = tf.keras.layers.Dense(32, activation="softmax")
```



Some of the most common types of activation functions.

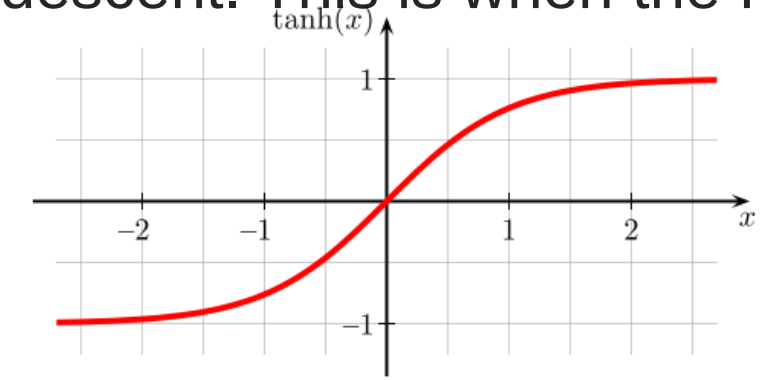
- **Hyperbolic tangent function (tanh)**
- The hyperbolic tangent function is a shifted version of the sigmoid version. Instead of squeezing the signal values between 0 and 1, tanh squishes all values to the range

–1 to 1.

- Tanh almost always works better than the sigmoid function in hidden layers because it has the effect of centring your data so that the mean of the data is close to zero rather than 0.5, which makes learning for the next layer a little bit easier:

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

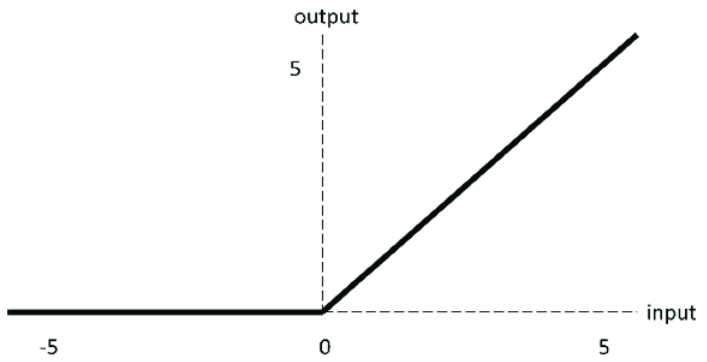
- One of the downsides of both sigmoid and tanh functions is that if (z) is very large or very small, then the gradient (or derivative or slope) of this function becomes very small (close to zero), which will slow down gradient descent. This is when the ReLU activation function.



Some of the most common types of activation functions.

- **Rectified linear unit**
  The rectified linear unit (ReLU) activation function activates a node only if the input is above zero. If the input is below zero, the output is always zero. But when the input is higher than zero, it has a linear relationship with the output variable. The ReLU function is represented as follows:
- f (x) = max (0, x)

- ReLU activation is that the derivative is equal to zero when (x) is negative.
- Leaky ReLU is a ReLU variation that tries to mitigate this issue. Instead of having the function be zero when x < 0, leaky ReLU introduces a small negative slope (around 0.01) when (x) is negative. It usually works better than the ReLU function, although it's not used as much in practice.
- f (x) = max(0.01x, x)
-



Summary

- For **hidden** layers—In most cases, you can use the **ReLU** activation function (or leaky ReLU) in hidden layers.

- For the output layer—The **softmax** activation function is generally a good choice for most **classification** problems when the classes are mutually exclusive.
- The **sigmoid** function serves the same purpose when you are doing **binary classification**.
- For **regression** problems, you can simply use **no activation function** at all, since the weighted sum node produces the continuous output that you need:
- for example, if you want to predict house pricing based on the prices of other houses in the same neighbourhood.

```python
#Dependencies
import keras
from keras.models import Sequential
from keras.layers import Dense
# Neural network
model = Sequential()
model.add(Dense(4, input_dim=3, activation="relu"))
model.add(Dense(5, activation="relu"))
model.add(Dense(10, activation="softmax"))
model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_3 (Dense)              (None, 4)                 16
_________________________________________________________________
dense_4 (Dense)              (None, 5)                 25
_________________________________________________________________
dense_5 (Dense)              (None, 10)                60
=================================================================
Total params: 101
Trainable params: 101
Non-trainable params: 0
```

```
1
2   #Dependencies
3   import keras
4   from keras.models import Sequential
5   from keras.layers import Dense
6   model = Sequential()
7   model.add(Dense(5, input_dim=3, activation= "relu"))
8   model.add(Dense(6, activation= "relu"))
9   model.add(Dense(1))
10  model.summary() #Print model Summary
```

```
Model: "sequential_3"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_9 (Dense)              (None, 5)                 20
_________________________________________________________________
dense_10 (Dense)             (None, 6)                 36
_________________________________________________________________
dense_11 (Dense)             (None, 1)                 7
=================================================================
Total params: 63
Trainable params: 63
Non-trainable params: 0
_________________________________________________________________
```

The feedforward process

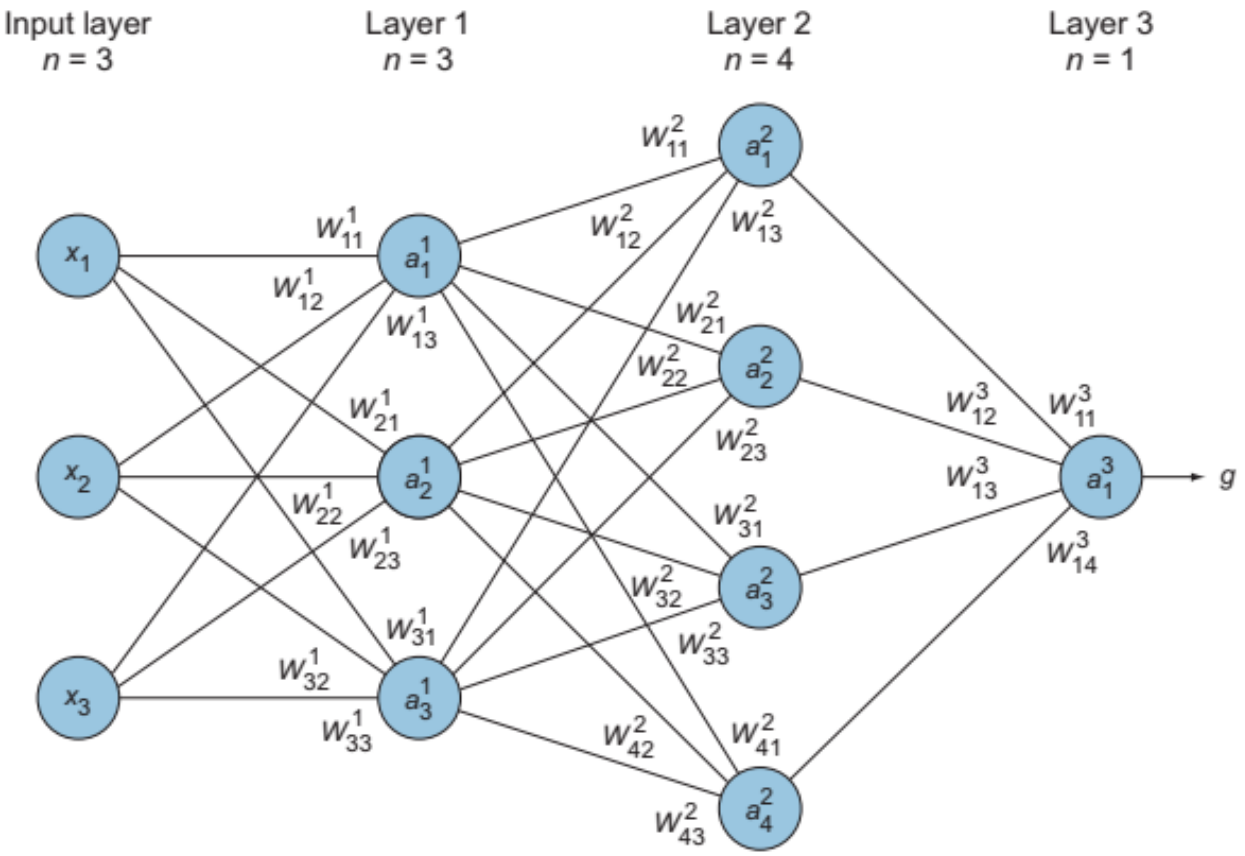
- The process of computing the linear combination and applying the activation function is called feedforward.
- The term feedforward is used to imply the forward direction in which the information flows from the input layer through the hidden layers, all the way to the output layer.
- This process happens through the implementation of two consecutive functions: **the weighted sum** and the **activation function**.

Layers—This network consists of an input layer with three input features, and three hidden layers with 3, 4, 1 neurons in each layer.

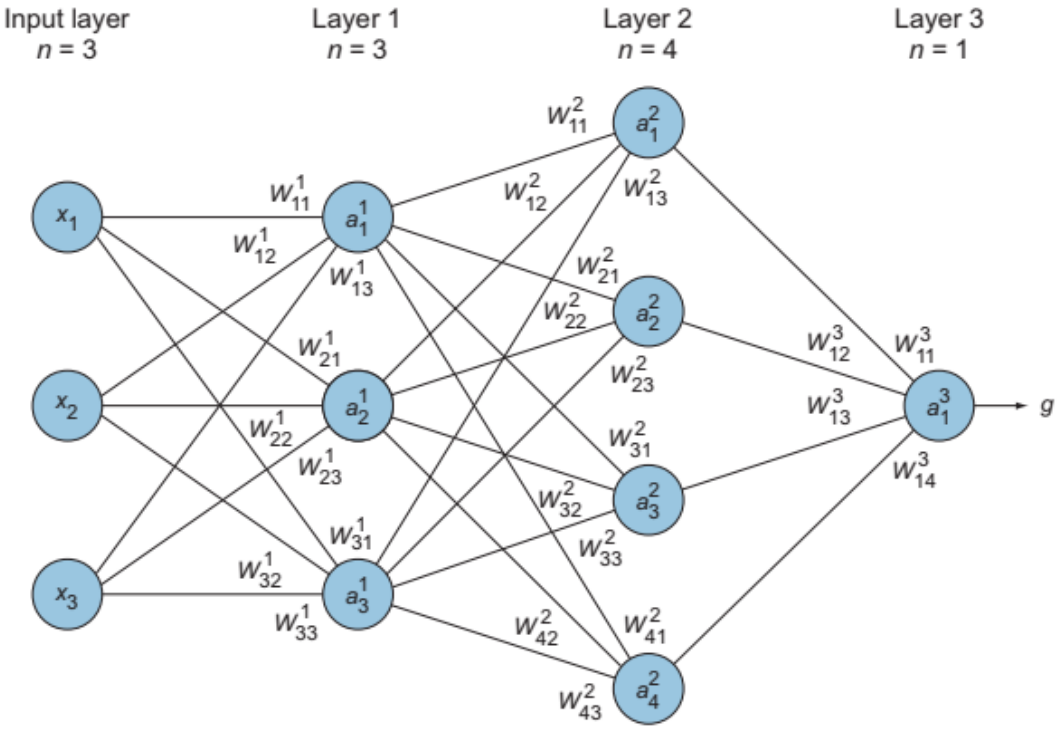
Weights and biases (w, b)—The edges between nodes are assigned random weights denoted as Wab (n), where (n) indicates the layer number and (ab) indicates the weighted edge connecting the ath neuron in layer (n) to the bth neuron in the previous layer (n – 1).

Activation functionsσ(x)—In this example, we are using the sigmoid function σ(x) as an activation function.

Node values (a)—We will calculate the weighted sum, apply the activation function, and assign this value to the node a_M^n, where n is the layer number and m is the node index in the layer. For example, a_2^3 means node number 2 in layer 3.

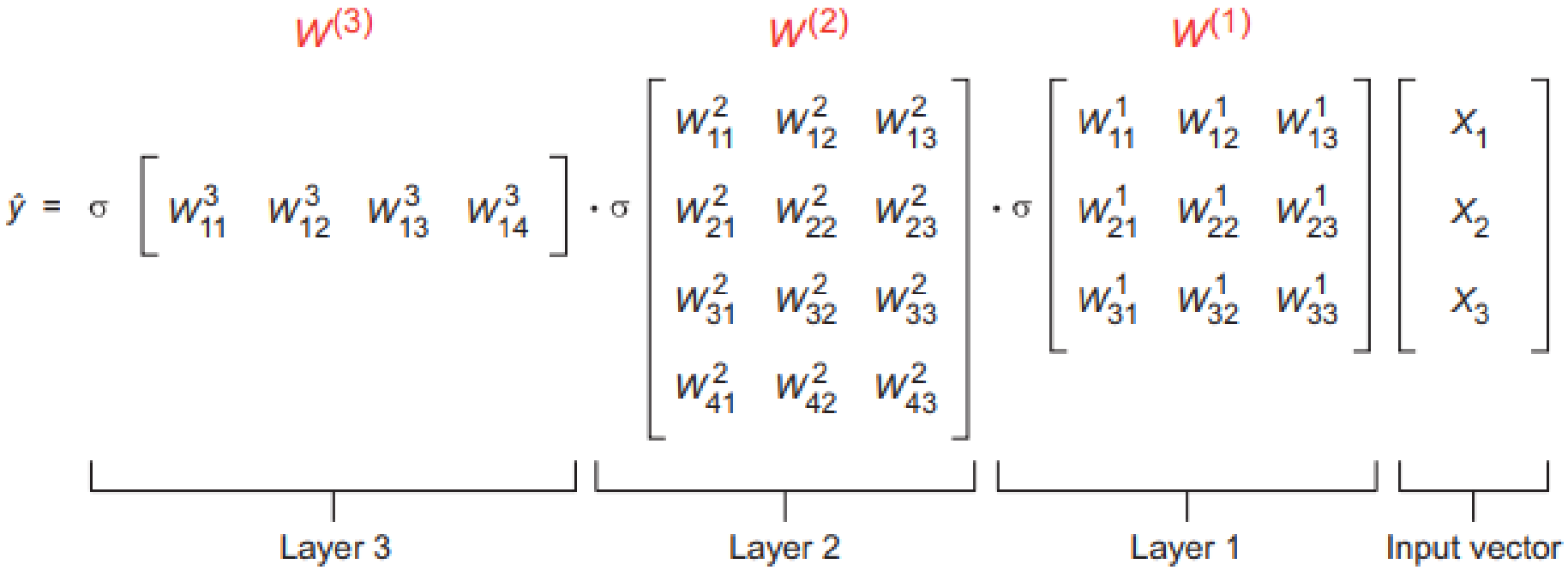


- $a^{(1)}_1 = \sigma(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + w_{31}^{(1)} x_3)$
- $a^{(1)}_2 = \sigma(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{32}^{(1)} x_3)$
- $a^{(1)}_3 = \sigma(w_{13}^{(1)} x_1 + w_{23}^{(1)} x_2 + w_{33}^{(1)} x_3)$
- .
- .
- .



- $a_1^{(2)} = \sigma\left(w_{11}^{(3)} a_1^{(2)} + w_{12}^{(3)} a_2^{(2)} + w_{13}^{(3)} a_3^{(2)} + w_{14}^{(3)} a_4^{(2)}\right)$

Optional


$W^{(3)}$
$W^{(2)}$
$W^{(1)}$
$\hat{y} = \sigma \begin{bmatrix} w_{11}^3 & w_{12}^3 & w_{13}^3 & w_{14}^3 \end{bmatrix} \cdot \sigma \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \\ w_{21}^2 & w_{22}^2 & w_{23}^2 \\ w_{31}^2 & w_{32}^2 & w_{33}^2 \\ w_{41}^2 & w_{42}^2 & w_{43}^2 \end{bmatrix} \cdot \sigma \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$
Layer 3
Layer 2
Layer 1
Input vector

Vectors and matrices refresher

If you understood the matrix calculations we just did in the feedforward discussion, feel free to skip this sidebar. If you are still not convinced, hang tight: this sidebar is for you.

The feedforward calculations are a set of matrix multiplications. While you will not do these calculations by hand, because there are a lot of great DL libraries that do them for you with just one line of code, it is valuable to understand the mathematics that happens under the hood so you can debug your network. Especially because this is very trivial and interesting, let's quickly review matrix calculations.

Let's start with some basic definitions of matrix dimensions:

- A *scalar* is a single number.
- A *vector* is an array of numbers.
- A *matrix* is a 2D array.
- A *tensor* is an *n*-dimensional array with $n > 2$.

Matrix dimensions: a scalar is a single number, a vector is an array of numbers, a matrix is a 2D array, and a tensor is an *n*-dimensional array.

We will follow the conventions used in most mathematical literature:

- Scalars are written in lowercase and italics: for instance, n.
- Vectors are written in lowercase, italics, and bold type: for instance, $\boldsymbol{x}$.
- Matrices are written in uppercase, italics, and bold: for instance, $\boldsymbol{X}$.
- Matrix dimensions are written as follows: (row × column).

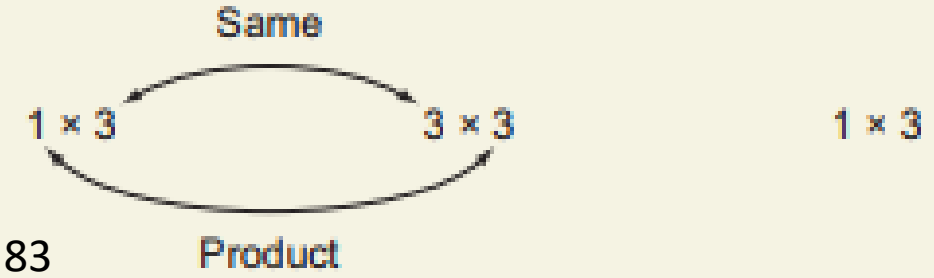
Multiplication:

- *Scalar multiplication*—Simply multiply the scalar number by all the numbers in the matrix. Note that scalar multiplications don't change the matrix dimensions:

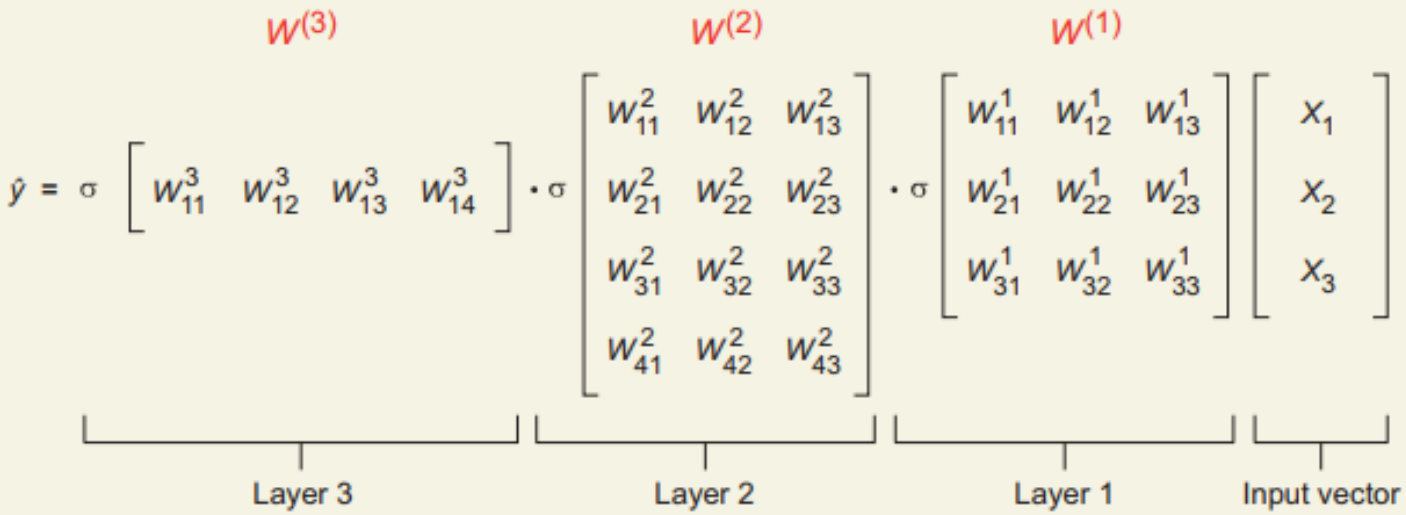
$$2 \cdot \begin{bmatrix} 10 & 6 \\ 4 & 3 \end{bmatrix} = \begin{bmatrix} 2 \cdot 10 & 2 \cdot 6 \\ 2 \cdot 4 & 2 \cdot 3 \end{bmatrix}$$

- *Matrix multiplication*—When multiplying two matrices, such as in the case of $(\text{row}_1 \times \text{column}_1) \times (\text{row}_2 \times \text{column}_2)$, column_1 and row_2 must be equal to each other, and the product will have the dimensions $(\text{row}_1 \times \text{column}_2)$. For example,

$$\begin{bmatrix} 3 & 4 & 2 \end{bmatrix} \cdot \begin{bmatrix} 13 & 9 & 7 \\ 8 & 7 & 4 \\ 6 & 4 & 0 \end{bmatrix} = \begin{bmatrix} x & y & z \end{bmatrix}$$



where $x = 3 \cdot 13 + 4 \cdot 8 + 2 \cdot 6 = 83$



The matrix equation from the main text. Use it to work through matrix dimensions.

The last thing I want you to understand about matrices is *transposition*. With transposition, you can convert a row vector to a column vector and vice versa, where the shape ($m \times n$) is inverted and becomes ($n \times m$). The superscript (A^T) is used for transposed matrices:

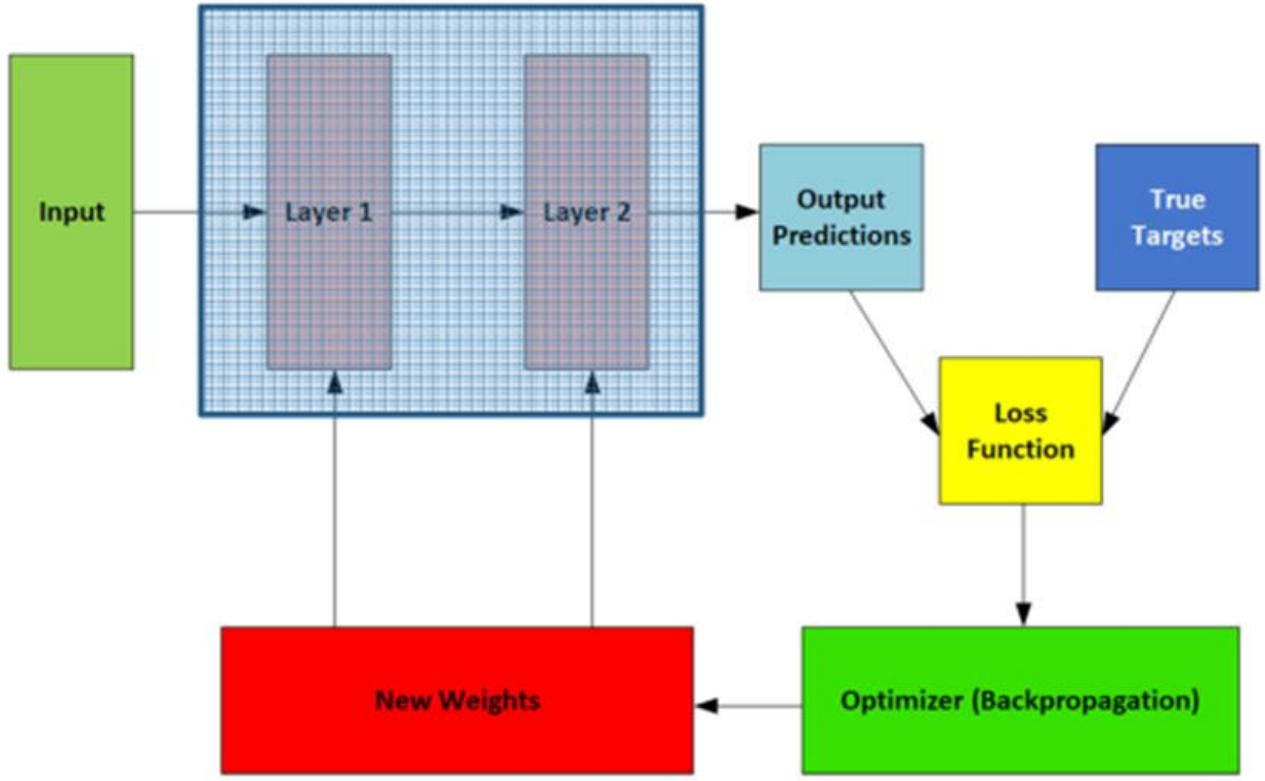
$$A = \begin{bmatrix} 2 \\ 8 \end{bmatrix} \quad \Rightarrow A^T = [2 \;\; 8]$$

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 \\ 2 & 4 \\ 1 & -1 \end{bmatrix} \quad \Rightarrow A^T = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 4 & -1 \end{bmatrix}$$

Anatomy of a neural network

- *Layers*, which are combined into a *network* (or *model*)
- The *input data* and corresponding *targets*
- The *loss function*, which defines the feedback signal used for learning
- The *optimizer*, which determines how learning proceeds



Assignment 1
Deep learning

Using Keras :Build Multi-Layer Perceptron Neural Network Models

- Input layer =4
- 3 hidden layers (Dense Layers) : the number of neurons in each hidden layer based on the count of your name letters example: mohammad Yousef mohammad (8 - 4 - 8)
- The model have to solve classification problem (to classify four classes).
- Calculate(manually) the number of trainable parameters in the model.